

Evaluation Design Quickstart

Smart Room, CARLA, and GroundingEval. CityOS Summer 2026

1 Evaluation as structure

Evaluation turns a vague sensing project into a sequence of **winnable questions**. In a multi-layered stack, bad evaluation at the sensing layer contaminates every layer above. If cameras are miscalibrated, detection accuracy numbers are meaningless. If timestamps drift, multi-view fusion produces garbage. If you do not log what changed between runs, you cannot explain why performance went up or down.

Your first milestone should be a **measurement system**, because model novelty means nothing without a way to score it. Aim for a visible artifact every two to three weeks.

- **By week 4.** Calibration report showing reprojection error for each camera, timestamp synchronization check across streams, and a documented room layout.
- **By week 6.** Baseline accuracy score on at least one scenario (even if the “model” is just YOLO out of the box or manual counting).
- **By week 8.** One clear improvement over baseline, with an ablation showing what change caused the improvement.

Good sequence for the summer: placement → calibration → sync validation → one tiny benchmark → baseline → one clear improvement → expand scenarios.

CityOS principle. LLMs generate, and deterministic checkers verify. GroundingEval checks TeLLMe answers against ground truth *you* define. Smart Room and CARLA teams supply scenarios and labels.

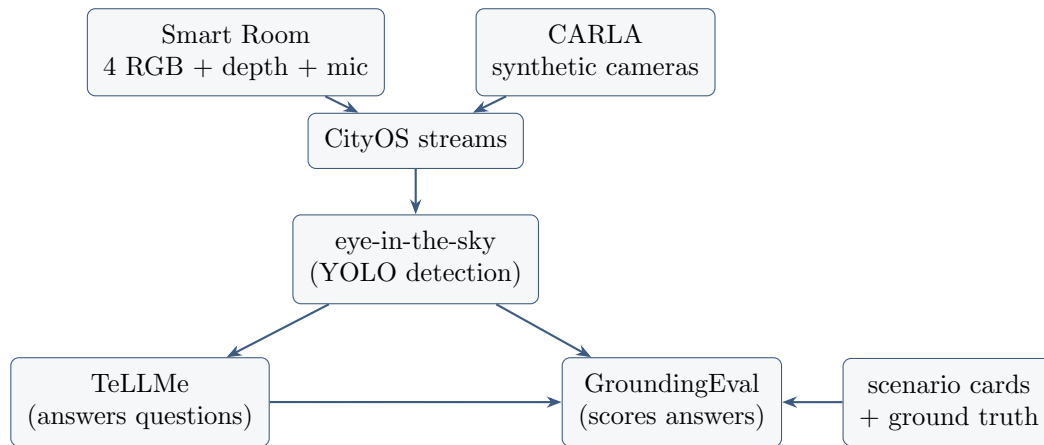
Answer-sheet model. GroundingEval writes the exam, including scenarios, questions, and verified answers. TeLLMe runs the test and scores against your sheet. You hand off the answer key; you do not grade TeLLMe yourself. LLMs may *draft* QA pairs to save time, but your team must verify every entry before it goes in the sheet.

2 Stack at a glance

The CityOS summer stack has five layers. Each layer has a question you must answer before moving up.

Layer	Question	If you skip it
Sensing	Did we record the right thing at the right time?	Every layer above is working with bad data.
Perception	Did the model infer the correct state from that data?	False confidence in downstream decisions.
Planning	Did the agent choose the right action sequence?	Good detections but failed tasks.
Control	Did execution stay safe and stable?	System looks correct on paper, fails live.
Logging	Can we compare runs and debug failures?	No reproducibility, no learning.

The diagram below shows how Smart Room and CARLA feed into CityOS, which runs **eye-in-the-sky** detection. TeLLMe answers questions about scenes, and GroundingEval scores those answers against your ground truth.



3 Six rules for rigorous evaluation

1. **Validity.** Measure what you actually claim. If your goal is room occupancy, face-detection AP alone is insufficient because faces can be occluded or turned away. You need a metric that captures occupancy directly.
2. **Repeatability.** Same scenario, same procedure, comparable runs. If you cannot reproduce a result from a clean checkout, you have learned nothing you can build on.
3. **Baselines first.** Before trying fusion or anything clever, run a trivial baseline (random guess, majority class) and a respectable baseline (off-the-shelf YOLO, simple heuristic). If you cannot beat the trivial baseline, something is broken. If you cannot beat the respectable baseline, your approach may need rethinking.
4. **One factor at a time.** If you change camera placement, annotation policy, training data, and model architecture simultaneously, you cannot attribute any improvement or degradation to a specific change. Change one factor, measure, then change the next.
5. **Freeze a benchmark version.** Lock room layout, calibration files, and annotation schema for each benchmark release. If you keep changing the test while tuning, your numbers are meaningless.
6. **Log everything that can change.** For every run, record calibration file version, code commit hash, random seed, data split, and hardware. Without this, you cannot debug regressions or explain improvements.

4 Smart Room workflow

4.1 Hardware (ordering Jun/Jul 2026)

The room will have four RGB cameras for appearance, identity, and multi-view coverage; depth camera(s) for geometry, occlusion reasoning, and distinguishing “on the chair” from “next to the chair”; and one microphone array for speech activity detection when wired. A dedicated GPU machine handles detection inference and stores recordings.

Until hardware is installed, start your scenario design and ground-truth workflow in CARLA (see next section).

4.2 Step-by-step workflow

1. **Freeze room layout for this benchmark version.** Document camera positions, angles, and any furniture placement. Take photos. If anything moves, you need a new benchmark version.

2. **Calibrate cameras and verify synchronization.** Run intrinsic calibration (checkerboard) for each camera. Run extrinsic calibration to get all cameras in the same coordinate frame. Verify that timestamps across streams are synchronized within acceptable tolerance (a few milliseconds for video, tighter for audio).
3. **Run a scripted scenario.** Start with something simple and observable. Person enters, sits at desk for 30 seconds, stands, exits. Script the actions so you know exactly what ground truth should be at each timestamp.
4. **Save artifacts to scenarios/<scenario_name>/.** Include raw video frames (or compressed video), detector JSON output from `eye-in-the-sky`, and a `metadata.json` with scenario name, start/end timestamps, camera IDs, and any notes about what happened.
5. **Write a scenario card and ground truth file.** The scenario card describes setup and actions in plain text. The ground truth file contains timestamped facts (at T=0:15, one person seated at desk; at T=0:45, room empty). Use the VQA format from `TEAM_3_GROUNDINGEVAL.pdf`.
6. **Run pipeline evaluation.** Measure detection latency (time from frame capture to detection JSON), missing-frame rate, and any obvious failures. This is your sensing-layer health check.
7. **Ship the bundle to GroundingEval.** Or, if you are also on the GroundingEval team, use YOLO detection output plus human verification to build ground truth directly. The key constraint is that every answer in the ground-truth file must be human-verified.

4.3 Who does what

Smart Room team sets up sensors, runs recordings, and produces raw data bundles. CARLA team exports simulator state as ground truth. GroundingEval designs scenarios jointly with both teams and publishes the final scenario, question, and answer tables that TeLLMe will be tested against.

5 CARLA as methodological mirror

Start here while Smart Room sensors are still arriving. One CARLA route or street is enough for now; do not over-engineer custom maps yet.

CARLA teaches repeatability and synchronized logging, which you then carry back to the room. The simulator gives you perfect ground truth (you know exactly where every actor is), deterministic replay (run the same scenario 100 times with identical results), and controlled ablations (change one weather parameter, measure the effect). Real acoustics, human behavior, and indoor semantics still require the physical space, but the discipline you learn in CARLA transfers directly.

5.1 Practical settings

Use **synchronous mode with a fixed timestep** when precision is required. CARLA documentation is explicit that this is the only way to get deterministic behavior. GPU camera sensors may lag by a frame or two, so align by frame number and timestamp rather than assuming streams are automatically synchronized.

Artifact format should match the room. Export `eye-in-the-sky`-compatible JSON (bounding boxes, class labels, track IDs) and ground-truth JSON (actor positions, states, timestamps). If both environments produce the same format, the same GroundingEval checkers work on both.

Prove a claim once in CARLA under clean control, then once in the room under realism. If performance differs significantly, that gap is itself a research finding worth documenting.

5.2 When to add difficulty

Only after TeLLMe scores near 100% on the *easy* tier (small scenes, few people, two or three sensors). Then add a medium tier (more people, intersections, more cameras) and a hard tier (many agents, complex multi-camera coordination). The research benchmark lives at the frontier where accuracy drops, so do not add difficulty until you have saturated the easier tiers.

	Smart Room	CARLA
Best for	Real occlusion, real human behavior, real acoustics	Repeatable scenario sweeps, debugging, ablations
Ground truth	Human labels from scenario cards	Simulator actor state (perfect)
Weak at	Exact repeatability	Indoor task semantics (driving simulator)

6 First two weeks checklist

Complete these tasks before the end of week 2 to confirm your workflow is operational.

- ☐ Read this handout (approximately 15 minutes).
- ☐ Skim `PROJECT_OVERVIEW.pdf` to understand the full CityOS stack diagram and where your team fits.
- ☐ Run one scripted room scenario (if hardware is available) *or* one CARLA replay. Save all artifacts to the standard directory structure.
- ☐ Write one scenario card describing setup, actions, and expected facts. Include five timestamped QA pairs (question, correct answer, timestamp). Verify each answer against the video or simulator state.
- ☐ Manually count people in 10 frames from your recording. Compare your count to **eye-in-the-sky** or YOLO JSON output. Note any discrepancies.
- ☐ Produce one run card documenting date, layout version, code commit, and what changed from any previous run.

Early win. One scenario, one ground-truth file, one baseline score. The “model” can be hand-counting. The point is to have a complete pipeline from data collection through scoring before you worry about improving the model.

7 Read these four papers (approximately 60 minutes total)

These papers are directly relevant to the summer work and accessible to undergraduates.

1. **Dosovitskiy et al., CARLA: An Open Urban Driving Simulator (CoRL 2017).** Read for layered evaluation of a modular pipeline. The paper reports success rate and infractions separately, which is exactly the structure you should use when evaluating TeLLMe. Focus on the evaluation sections; skip the model architecture details.
<https://arxiv.org/abs/1711.03938>
2. **Milan et al., MOT16: A Benchmark for Multi-Object Tracking (2016).** Read for benchmark design. The paper shows how standardized data, annotations, and evaluation protocols create a shared scientific target. This is the template for how you should think about publishing your scenario and ground-truth datasets.

<https://arxiv.org/abs/1603.00831>

3. **Roth et al., AVA-ActiveSpeaker: An Audio-Visual Dataset for Active Speaker Detection (2019).** Read for audio-visual room-scale evaluation. The task (who is speaking?) is intuitive, the official evaluation procedure is public, and the paper demonstrates how to evaluate fusion against single-modality baselines.

<https://arxiv.org/abs/1807.10013>

4. **Shojaee et al., The Illusion of Thinking (Apple ML Research, 2025).** Read for why eval design for LLMs requires controlled complexity. The paper shows that reasoning models collapse beyond a complexity threshold and that apparent “thinking” can be superficial. Directly relevant to GroundingEval: your tiered benchmarks (easy, medium, hard) should probe where TeLLMe’s reasoning breaks down.

<https://arxiv.org/abs/2506.06941>

Optional for calibration owners. Zhang, *A Flexible New Technique for Camera Calibration* (IEEE PAMI 2000). Standard reference for intrinsic and extrinsic calibration from planar targets. Search “Zhang camera calibration” or see [Microsoft Research](#).